

HRMS Leave Management

Leave Balance, Carry Forward & Rollback

Technical & Business Documentation

A practical guide for developers, testers, HR users, managers and non-technical stakeholders.

Scope

This document explains the current Leave Balance, Carry Forward and Rollback implementation, including business rules, formulas, APIs, examples, database behavior, testing and architecture.

Technology

ASP.NET Core • MediatR • Clean/Modular Architecture • EF Core • SQL Server

Version: Carry Forward V1 | Development API examples use <http://localhost:5168>

1. Executive Summary

The feature answers three simple questions: how much leave was allocated, how much was brought from the previous year, and how much remains usable.

- Entitled Days — normal annual allocation.
- Carry Forward Days — unused days brought from the previous year.
- Used Days — days consumed by approved leave.
- Available Days — what the employee can still use.

Final balance formula:

$\text{AvailableDays} = \text{EntitledDays} + \text{CarryForwardDays} - \text{UsedDays}$

Carry Forward is policy-driven. CL/PL/SL/EL values are stored in LeavePolicyRules rather than hard-coded in the calculation.

The V1 implementation is transactional and idempotent: a failed generation rolls back, and rerunning generation does not create duplicate active entitlement records.

2. Key Terms in Simple Language

Term	Meaning
Leave Type	Category such as CL, PL, SL or EL.
Leave Year	The leave/financial period, such as FY2026 or FY2027.
Leave Policy	Policy selected for a company.
Leave Policy Rule	Rule for one leave type, including annual entitlement and carry-forward settings.
Employee Leave Entitlement	One employee's allocation for one leave type in one leave year.
Entitled Days	Normal annual allocation.
Used Days	Days consumed by approved leave.
Carry Forward Days	Unused days brought from the previous year.
Available Days	Days still available to the employee.
Soft Delete	Record remains in the database but IsDeleted is set to 1.
Idempotent	Repeating the same operation does not create another effective result.

3. Entitlement Data Model

One EmployeeLeaveEntitlement represents one employee + one leave year + one leave type.

EmployeeLeaveEntitlement

EmployeeId
LeaveYearId
LeaveTypeId
LeavePolicyRuleId
EntitledDays
CarryForwardDays
UsedDays
IsDeleted
CreatedOn / ModifiedOn / DeletedOn

The active combination of EmployeeId + LeaveYearId + LeaveTypeId is protected by a unique filtered index, helping prevent duplicate active entitlement records.

4. Current Carry Forward Policy

Leave Type	Annual	Carry Forward	Maximum
CL	12	No	0

Leave Type	Annual	Carry Forward	Maximum
PL	10	Yes	5
SL	12	No	0
EL	15	Yes	10

Policy fields are `IsCarryForwardAllowed` and `MaximumCarryForwardDays`. The calculation reads these values from `LeavePolicyRule`.

5. Leave Balance

The Balance API reads active entitlements for the requested employee and leave year. Supplying `leaveTypeId` returns one leave type; omitting it returns all active leave types for that year.

$\text{AvailableDays} = \text{EntitledDays} + \text{CarryForwardDays} - \text{UsedDays}$

Example: 12 entitled, 0 carry forward, 3 used → 9 available.

Example with carry forward: 10 entitled, 5 carried, 2 used → 13 available.

6. Balance API

All leave types:

GET `/api/EmployeeLeaveEntitlement/{employeeId}/{leaveYearId}/balance`

One leave type:

GET `/api/EmployeeLeaveEntitlement/{employeeId}/{leaveYearId}/balance?leaveTypeId={leaveTypeId}`

Representative response:

```
{
  "employeeId": "19ed1df1-c3cc-4614-85f0-08def7b598a7",
  "leaveYearId": "8bb8836a-3920-4da2-41cf-08defd305cc8",
  "leaveTypeId": "9a5ff5c9-49df-43f4-5107-08defd337f69",
  "entitledDays": 12.00,
  "carryForwardDays": 0.00,
  "usedDays": 3.00,
  "availableDays": 9.00
}
```

7. What Happens When Leave Is Approved?

The existing approval flow updates LeaveRequest and EmployeeLeaveEntitlement in the same transaction. Approval of 3 days increases UsedDays by 3.

Before approval:
Entitled = 12
CarryForward = 0
Used = 0
Available = 12

After approval of 3 days:
Entitled = 12
CarryForward = 0
Used = 3
Available = 9

A valid cancellation restores the used balance by decreasing UsedDays according to the existing transaction logic.

8. Carry Forward Business Rule

At the end of a leave year, unused leave may move to the next year only when the policy permits it.

$\text{ClosingBalance} = \text{MAX}(0, \text{EntitledDays} - \text{UsedDays})$

If carry-forward is not allowed:
CarryForwardDays = 0

If allowed:
 $\text{CarryForwardDays} = \text{MIN}(\text{ClosingBalance}, \text{MaximumCarryForwardDays})$

9. Carry Forward Examples

Full PL carry forward

Item	Value
Annual entitlement	10
Used	0
Closing balance	10
Maximum carry forward	5
Carry forward	5
Next-year available	15

Partial PL carry forward

Item	Value
Annual entitlement	10
Used	7
Closing balance	3
Maximum carry forward	5
Carry forward	3
Next-year available	13

CL no carry forward

Item	Value
Annual entitlement	12

Item	Value
Used	2
Closing balance	10
Carry allowed	No
Carry forward	0

10. Timing Decision

V1 does not permanently calculate final carry-forward before the current year closes. The recommended process is to close the previous year first, then generate the next year's entitlements.

```

FY2026
01-Apr-2026 → 31-Mar-2027
  |
  v
FY2026 closes
  |
  v
Generate FY2027
  |
  v
Calculate actual FY2026 closing balances
  |
  v
Create FY2027 entitlements

```

This prevents a late approved leave from making a previously calculated carry-forward value incorrect.

11. Generate Carry Forward API

POST /api/EmployeeLeaveEntitlement/carry-forward

Request:

```

{
  "leaveYearId": "8bb8836a-3920-4da2-41cf-08defd305cc8"
}

```

The caller supplies only the target LeaveYearId. The handler finds the previous Leave Year for the same company, reads previous entitlements and policy rules, calculates carry-forward and creates missing target-year entitlements.

Success response:

```

{
  "message": "Leave carry forward generated successfully."
}

```

12. Generation Logic

```

Target Leave Year
  |
  v
Find previous Leave Year
(same company; latest year ending before target start)
  |
  v
Read previous entitlements
  |
  v
Read LeavePolicyRule
  |
  v
Closing = MAX(0, Entitled - Used)
  |
  v
Carry = MIN(Closing, Maximum) when allowed
  |
  v
Create target entitlement

```

Entitled = Policy AnnualEntitlement
CarryForward = calculated value
Used = 0

13. Idempotency

Before creating a target entitlement, the handler checks for an active record with the same EmployeeId + LeaveYearId + LeaveTypeId. Existing active records are skipped.

Run 1:
PL → created with CarryForwardDays = 5

Run 2:
PL already exists → skipped

Result:
No duplicate active record and no addition of 5 + 5.

This was tested successfully with the current development data.

14. Transaction Safety

The generation uses the existing IUnitOfWorkTransaction. If processing fails before commit, the entire operation is rolled back.

```
BEGIN TRANSACTION
|
+-- read target year
+-- find previous year
+-- calculate
+-- add missing records
|
+-- error -> ROLLBACK
|
+-- success -> COMMIT
```

15. Rollback

V1 rollback is a business rollback, not a database transaction rollback after the original transaction has already committed. It uses the existing soft-delete pattern.

Rules:

- Only active target-year entitlements with CarryForwardDays > 0 are candidates.
- If a candidate has UsedDays > 0, rollback is rejected.
- If UsedDays = 0, the candidate is soft-deleted.
- Leave types with CarryForwardDays = 0 are not touched by this rollback operation.

16. Rollback API

POST /api/EmployeeLeaveEntitlement/carry-forward/rollback

Request:

```
{
  "leaveYearId": "8bb8836a-3920-4da2-41cf-08defd305cc8"
}
```

Success response:

```
{
  "message": "Leave carry forward rolled back successfully."
}
```

Soft delete means the record remains in SQL Server with IsDeleted = 1 rather than being physically deleted.

17. Full Tested Lifecycle

```
Generate
|
v
FY2027 carry-forward records created
|
v
Rollback
|
v
Carry-forward records soft-deleted
|
v
Generate again
|
v
New active records recreated
```

This lifecycle was tested successfully in the development environment.

18. Important V1 Rollback Limitation

The current rollback identifies candidates by target LeaveYearId and CarryForwardDays > 0. There is not yet a dedicated CarryForwardRun ID linking a specific execution to its generated records.

A future enhancement can add:

```
CarryForwardRun
-----
Id
FromLeaveYearId
ToLeaveYearId
ExecutedOn
Status
```

That would provide stronger operational audit and allow a specific generation run to be targeted for rollback. It is not required for the current V1.

19. SQL Verification

Find active carry-forward records:

```
SELECT
    e.Id,
    e.EmployeeId,
    lt.Code AS LeaveType,
    e.EntitledDays,
    e.CarryForwardDays,
    e.UsedDays,
    e.IsDeleted
FROM EmployeeLeaveEntitlements e
INNER JOIN LeaveTypes lt
    ON lt.Id = e.LeaveTypeId
WHERE e.LeaveYearId =
    '8BB8836A-3920-4DA2-41CF-08DEFD305CC8'
    AND e.CarryForwardDays > 0
    AND e.IsDeleted = 0
ORDER BY e.EmployeeId, lt.Code;
```

Calculate one employee's available balance:

```
SELECT
    lt.Code AS LeaveType,
    e.EntitledDays,
    e.CarryForwardDays,
    e.UsedDays,
    e.EntitledDays
    + e.CarryForwardDays
    - e.UsedDays AS AvailableDays
FROM EmployeeLeaveEntitlements e
INNER JOIN LeaveTypes lt
    ON lt.Id = e.LeaveTypeId
WHERE e.EmployeeId =
    '5D09DE86-0892-44E3-FEB0-08DEECB71347'
    AND e.LeaveYearId =
    '8BB8836A-3920-4DA2-41CF-08DEFD305CC8'
    AND e.IsDeleted = 0
ORDER BY lt.Code;
```

20. SQL Stored Procedure — Personal Use

A SQL Server stored procedure was prepared as a manual/testing utility. It is not the official application implementation. The official application path remains Controller → MediatR → C# handler → repository → SQL Server.

The procedure follows the same calculation and idempotency rules. It should be treated as an administrative/test utility and reviewed if the application business rules change.

21. Test Matrix

Scenario	Expected
CL, unused	Carry = 0
SL, unused	Carry = 0
PL, 10 entitled, 0 used, max 5	Carry = 5
PL, 10 entitled, 7 used, max 5	Carry = 3
EL, 15 entitled, 0 used, max 10	Carry = 10
Run generation twice	No duplicate active record
Generation error	Whole transaction rolls back
Rollback unused carry-forward	Soft delete allowed
Rollback after usage	Rejected
Generate after rollback	New active record can be created
Balance with carry-forward	Entitled + Carry - Used

22. Layperson End-to-End Example

An employee gets 10 PL days, uses 7, and has 3 remaining. The policy allows carry forward up to 5. Because only 3 remain, the employee carries 3—not 5.

Old year:
Given = 10
Used = 7
Remaining = 3

Policy maximum = 5

New year:
Annual = 10
Carry Forward = 3
Used = 0

Available = $10 + 3 - 0 = 13$

The policy sets the ceiling, but the employee's actual closing balance determines how much is carried.

23. Architecture

```
Controller
|
v
MediatR Command
|
v
Application Handler
|
+--> IReadRepository
|   +--> LeaveYear
|   +--> EmployeeLeaveEntitlement
|   +--> LeavePolicyRule
|
+--> IWriteRepository
|
+--> IUnitOfWorkTransaction
|
v
SQL Server
```

The implementation stays inside the existing Leave module. V1 does not require a new module, repository abstraction, LeaveRequest allocation table, or new Unit of Work abstraction.

24. Completion Checklist

Capability	Status
Carry-forward policy fields	Completed
Employee CarryForwardDays	Completed
Balance formula and API	Completed and tested
Full carry-forward	Tested
Partial carry-forward	Tested
Maximum rule	Tested
No-carry leave types	Tested
Idempotent generation	Tested
Transactional generation	Implemented
Soft-delete rollback	Tested
Generate after rollback	Tested
CarryForwardRun audit entity	Future enhancement

25. Quick Reference

Operation	Endpoint
Get all balances	GET /api/EmployeeLeaveEntitlement/{employeeId}/{leaveYearId}/balance
Get one balance	GET /api/EmployeeLeaveEntitlement/{employeeId}/{leaveYearId}/balance?leaveTypeId={leaveTypeId}
Generate carry forward	POST /api/EmployeeLeaveEntitlement/carry-forward
Rollback carry forward	POST /api/EmployeeLeaveEntitlement/carry-forward/rollback

Sample IDs in this document are from the current development/test data. Replace them for other environments.